AASHIQ SAYYED (1002082459)
ALFRED JOMMA (1002089552)
SRIKANTH PUTNALA (1002037928)

BRIEF DESCRIPTION

Consider we are given a set of grocery stores where the head of all stores wants to query about the available sanitizers inventory at all stores to move inventory store to store to make balance over the quantity of sanitizers inventory at all stores. The task is performed by a single transaction **T** that's component $\mathbf{T_n}$ at the $\mathbf{n^{th}}$ store and a store $\mathbf{S_0}$ corresponds to $\mathbf{T_0}$ where the manager is located. The following sequence of activities are performed by **T** are below:
a) Component of transaction **(T)** $\mathbf{T_0}$ is created at the head-site(head-office).
b) $\mathbf{T_0}$ sends messages to all the stores to order them to create components $\mathbf{T_i}$**.**
c) Every $\mathbf{T_i}$ executes a query at the store "**i"** to discover the quantity of available sanitizers inventory and reports this number to $\mathbf{T_o}$**.**
d) Each store receives instruction and updates the inventory level and makes shipments to other stores where required.

But there are some problems that we may face during the execution of this process:
1) Atomicity property may be violated because any store **(**$\mathbf{S_n}$**)** may be instructed twice to send the inventory that may leave the database in an inconsistent state. To ensure atomicity property Transaction **T** must either commit at all the sites, or it must abort at all sites.
2) However, the system at store $\mathbf{T_n}$ may crash, and the instructions from $\mathbf{T_0}$ are never received by $\mathbf{T_n}$ because of any network issue and any other reason. So, the question arises what will happen to the distributed transaction running whether it abort or commit? Whether it recover or not?

Two-Phase Commit Protocol: This protocol is designed with the core intent to resolve the above problems, consider we have multiple distributed databases which are operated from different servers(sites) let's say $\mathbf{S_1, S_2, S_3, ...S_n}$**.** Where every $\mathbf{S_i}$ made to maintains a separate log record of all corresponding activities and the transition **T** has also been divided into the sub transactions $\mathbf{T_1, T_2, T_3,}$

...., T_n and each **T_i** are assigned to **S_i.** This is all maintained by a separate transaction manager at each **S_i.** We assigned anyone site as a **Coordinator.**

Some points to be considered regarding this protocol:

a) In a two-phase commit, we assume that each site logs actions at that site, but there is no global log.

b) The coordinator (Ci) plays a vital role in doing confirmation whether the distributed transaction would abort or commit.

c) In this **protocol** messages are made to be sent between the **coordinator (C_i)** and the other **sites.** As each message is sent, its logs are noted at each sending site, to aid in recovery should it be necessary.

The two phases of this protocol are as follow:

Phase-1st–

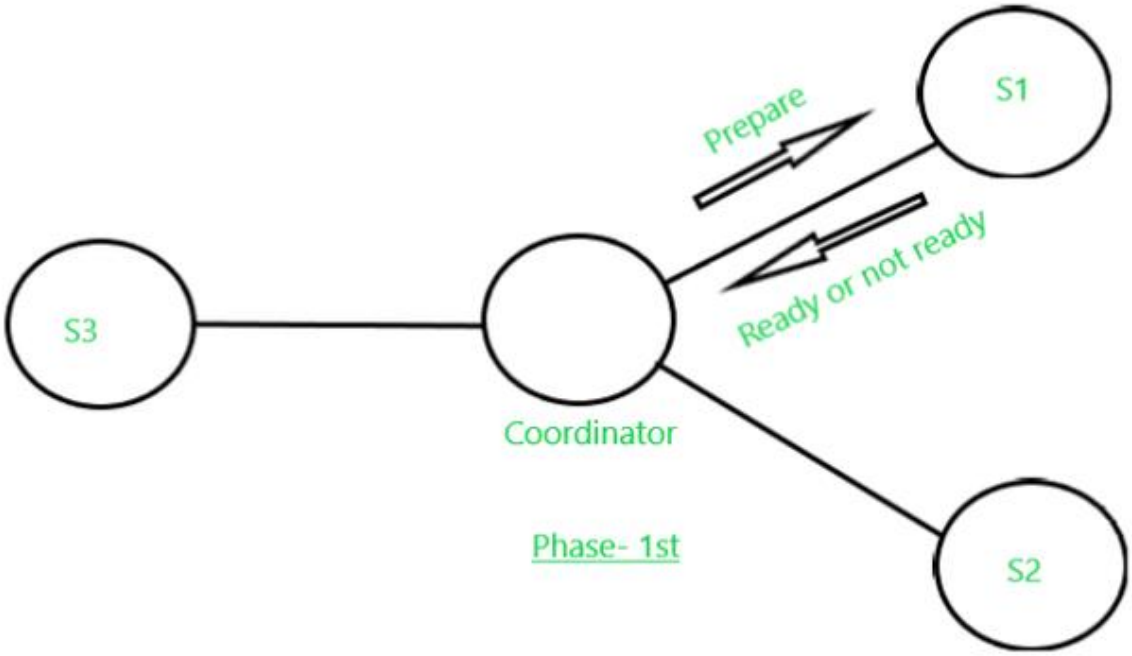
a) Firstly, the coordinator (C_i) places a log record **<Prepare T>** on the log record at its site.

b) Then, the **coordinator (C_i)** sends a **Prepare T** message to all the sites where the transaction(T) was executed.

c) Transaction manager at each site on receiving this message **Prepare T** to decide whether to commit or abort its component(portion) of T. The site can be delayed if the component has not yet completed its activity but must eventually send a response.

d) If the site doesn't want to commit, so it must write on **log record <no T>,** and local Transaction manager sends a message **abort T** to **C_i.**

e) If the site wants to commit, it must write on log record **<ready T>**, and local Transaction manager sends a message **ready T** to **C_i. Once the** ready **T message at C_i is sent** nothing can prevent it from committing its portion of **transaction T** except **Coordinator (C_i).**



Messaging in Phase- 1st

Phase- 2nd–

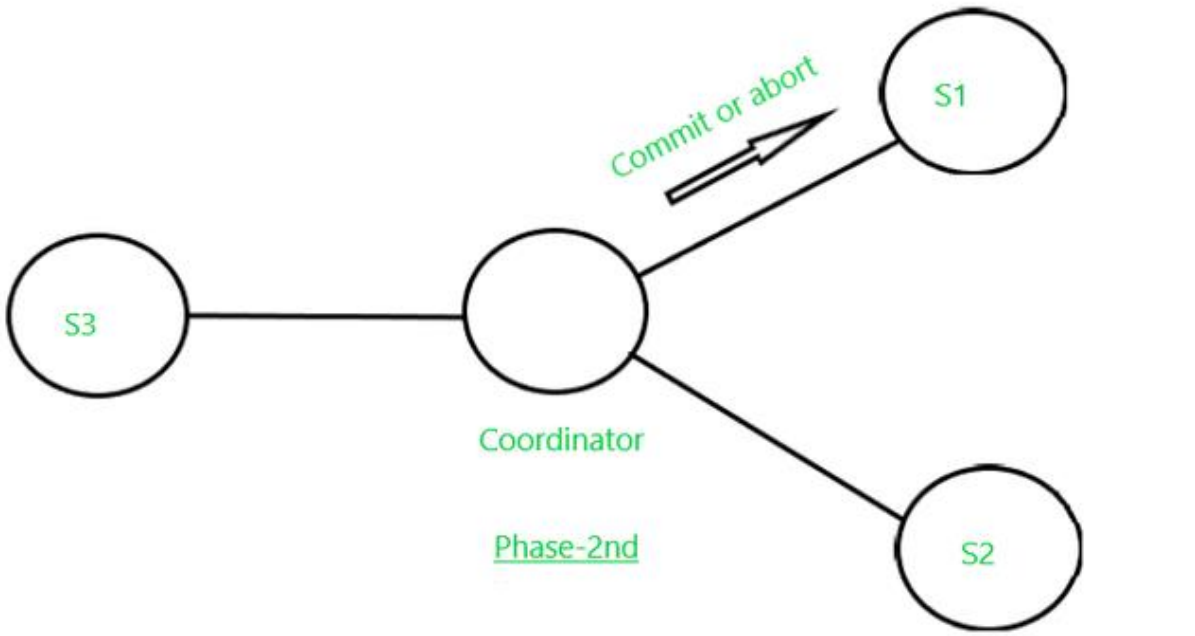
The Second phase started as the response abort T or commit T receives by the **coordinator (C_i) from all the sites that** are collaboratively **executing the transaction T.** However, it is possible that some sites fail to respond; it may be down, or it has been disconnected by the network. In that case, after a suitable timeout period will be given, after that time it will treat the site as if it had sent **abort T**. The fate of the transaction depends upon the following points:

a) If the coordinator receives **ready T** from all the participating sites of T, then it decides to **commit T**. Then, the coordinator writes on its site log record **<Commit T>** and sends a message **commit T** to all the sites involved in T.

b) If a site receives a **commit T** message, it commits the component of T at that site, and write it in log records **<Commit T>**.

c) If a site receives the message **abort T**, it aborts T and writes the log record **<Abort T>.**

d) However, if the coordinator has received **abort T** from one or more sites, it logs **<Abort T>** at its site and then sends **abort T** messages to all sites involved in transaction T.



Messaging in Phase- 2nd

Disadvantages:

a) The major disadvantage of the Two-phase commit protocol is faced when the coordinator site failure may result in blocking, so a decision either to commit or abort **Transaction(T)** may have to be postponed until coordinator recovers.

b) Blocking Problem: Consider a scenario, if a **Transaction(T)** holds locks on data-items of active sites, but amid the execution, if the coordinator fails and the active sites keep no additional log-record except **<readt T>** like **<abort>** or **<commit>.** So, it becomes impossible to determine what decision has been made (whether to **<commit> /<abort>**). So, in that case, the final decision is delayed until the **coordinator** is restored or fixed. In some cases, this may take a day or long hours to restore and during this time period, the locked data items remain inaccessible for other **transactions (T_i).** This problem is known as **Blocking Problem.**